Table 3-1. *Arduino Data Types*

Name	Size	Range
boolean	1 bit	true or false
byte	8 bit	0 to 255
char	8 bit	-128 to 127
int	16 bit	-32,768 to 32,767
long	32 bit	-2,147,483,648 to 2,147,483,647
float	32 bit	-3.4028235E+38 to 3.4028235E+38

Variable Qualifiers

As discussed earlier, variables will behave in certain ways based on the properties of their data type. By using **variable qualifiers**, it is possible to modify the behavior of certain variables in ways that will fundamentally change how we work with them. For example, the `const` or **constant** variable qualifier makes the value of a variable a read-only constant that cannot be changed once the value has been assigned. This could be useful in our sketch, so instead of the variable declaration reading as follows:

```
int red = 9;
```

We could write the line like this:

```
const int red = 9;
```

We use the `const` keyword at the beginning of a variable declaration to define a variable as a constant value to prevent us from inadvertently changing the variable's value later in the code. This is useful if we know that, as in the example, the pin number of the red LED will never change. If we attempt to assign a new value to the constant variable `red`, we will be reminded with a friendly compiler error. The constant qualifier is also useful for reducing the amount of memory that is needed by the code while it's running. Don't worry too much about that now; we will talk more about memory usage later in Chapter 8.

You might find examples of code that use the `#define` preprocessor directive instead of `const`, as in the line `#define red 9`. The constant qualifier is generally the preferred method of the Arduino development team over the `#define` preprocessor directive for defining constant variables. This is because the constant qualifier can be checked for errors and will follow all of the rules of variables, including variable scope defined momentarily, while not creating any potential conflicts with the Arduino library.

As briefly mentioned before, certain data types are either signed or unsigned. To modify the range of a few of the data types we could use the **unsigned** qualifier and the `unsigned` keyword, making them store positive-only values. This qualifier is beneficial for increasing the positive range of variables or for indicating to the reader of the code that the value expected will never be negative. For example:

```
unsigned int maxThreshold = 1024;
```